

AML Assignment — Submission Package

This directory is the **complete submission** for the UTM Big Data & AI Competition AML assignment. It contains the AML Knowledge Library (Task 1), the bad-actor detection pipeline and model outputs (Task 2), and model output explanations with an optional viewer (Task 3). All commands below are run from **this directory** (`AML_Submission/`).

For graders: Task 2 outputs → `task_2_model_outputs/`. Task 3 explanations → `task_3_model_outputs_explanations/`. To reproduce: place `master_features_with_clustering.csv` in `aml_detection_pipeline/data/input/`, then `pip install -r requirements.txt` and `python run_pipeline.py`. See **Submission checklist** below for every deliverable.

Table of contents

- [Main entry points: the three run scripts](#)
 - [What's in this package](#)
 - [Clustering](#)
 - [ML algorithms — playground for ML engineers](#)
 - [Setup and run](#)
 - [How the pipeline works](#)
 - [Submission checklist \(for graders\)](#)
 - [Design and methodology \(summary\)](#)
 - [Extensions and traceability](#)
-

Main entry points: the three run scripts

Everything you need to run from this package goes through three scripts. Run them from this directory (`AML_Submission/`):

Script	What it does
<code>run_pipeline.py</code>	Runs the full detection pipeline: rule-based scoring, fusion with all ML scores in <code>aml_detection_pipeline/ml_algorithms/outputs/</code> , predictions (top 5% flagged), and Task 2/3 outputs (<code>model_output_*.csv</code> and explanations). Writes to <code>aml_detection_pipeline/data/output/</code> and copies to <code>task_2_model_outputs/</code> and <code>task_3_model_outputs_explanations/</code> . Run this first to produce deliverables.
<code>run_viewer.py</code>	Launches the explanation viewer GUI so you can browse and search model outputs and explanations (paginated, search, filter by flagged, dark/light theme). Use after running the pipeline at least once.
<code>run_aml_library.py</code>	Opens the AML Knowledge Library (Task 1) in your default browser — red flags, typologies, and source traceability. Standalone; no pipeline run required.

There are no other top-level entry points. All other commands (e.g. `clean_data` scripts, model scripts inside `ml_algorithms`) are supporting; the three `run_*.py` scripts are the ones to use for running the pipeline, viewing results, and opening the library.

What's in this package

Component	Location	Description
Task 1: AML Knowledge Library	<code>task_1_aml_library/</code>	Red flags, indicators, and typologies (OCG-focused) with full source traceability. See <code>00_COMPREHENSIVE_RED_FLAGS_MASTER.md</code> , documents 01–08, <code>Red_Flag_to_Feature_Mapping.md</code> , and <code>SOURCES.md</code> .
Task 1 (optional): Web app	<code>task_1_aml_library/AML_Library_Web/</code>	Interactive browser for the Knowledge Library.
Tasks 2 & 3: Detection pipeline	<code>aml_detection_pipeline/</code>	Rule-based scoring + ML fusion, model outputs, and non-technical explanations. See pipeline <code>README.md</code> and <code>docs/CLIENT_CONTRACT.md</code> .
Input preparation	<code>clean_data/</code>	Produces <code>master_features_with_clustering.csv</code> used by the pipeline.
Clustering	<code>clustering/</code>	Clustering analysis and scripts (e.g. individual/business K-means) that produce cluster assignments. Output such as <code>Kmeans_Seperate.csv</code> (customer_id, cluster) is used by <code>clean_data</code> (e.g. script <code>11_merge_clustering.py</code>) to build the final pipeline input.
ML algorithms (model playground)	<code>aml_detection_pipeline/ml_algorithms/</code>	Intended playground for machine learning engineers : add or swap anomaly/detection models here. Each model lives in its own <code>model_*/</code> folder; as long as a model exports the right output (see below), the pipeline picks it up automatically. No pipeline code changes required.
Task 2 outputs (per model)	<code>task_2_model_outputs/</code>	Created when the pipeline runs; copies of each <code>model_output_*.csv</code> .

Component	Location	Description
Task 3 outputs (per model)	<code>task_3_model_outputs_explanations/</code>	Created when the pipeline runs; copies of each <code>model_output_*_explanations.csv</code> .

Clustering (`clustering/`)

This directory holds the **clustering work** (notebooks, cleaned scripts, and final outputs) that assigns each customer to a behavioural cluster (e.g. individual vs business, and sub-segments). The pipeline's rule-based scorer uses these cluster labels (e.g. high-risk clusters) from the merged feature file. The clustering output expected by the rest of the repo is a CSV with `customer_id` and `cluster` (string labels); `clean_data` merges this into `master_features_with_clustering.csv` via script `11_merge_clustering.py`. See `clean_data/README.md` and `clean_data/clustering/` for where the merge reads from.

ML algorithms — playground for ML engineers (`aml_detection_pipeline/ml_algorithms/`)

The pipeline designer intends **machine learning engineers to use this directory as a playground** to add, experiment with, and store detection models. You can add new `model_<name>/` folders (with your own scripts, notebooks, and artifacts) without changing any pipeline code. The pipeline **discovers** every CSV in `ml_algorithms/outputs/` whose name matches `scores_*.csv` and uses it in fusion and explanation generation.

Export contract (all that's required):

- **Filename:** `scores_<model>.csv` in `aml_detection_pipeline/ml_algorithms/outputs/` (e.g. `scores_my_LOF.csv`).
- **Columns:** `customer_id` (same set as the pipeline input) and `score` (one numeric column; higher = more anomalous/risky). The pipeline accepts column names `score`, `scores`, `risk_score`, or `outlier_score_01`; using `score` is preferred.
- **Rows:** One per customer in the pipeline input; same customer set.

Once a model writes a file that meets this contract into `outputs/`, the next run of `run_pipeline.py` will pick it up, fuse it with the rule-based score, and produce Task 2 and Task 3 outputs for that model. See `aml_detection_pipeline/ml_algorithms/README.md` and `aml_detection_pipeline/docs/CLIENT_CONTRACT.md` for full details.

Setup and run

Prerequisites: Python 3.8+ and the input feature file (see below).

1. **Environment** (from this directory):

```
pip install -r requirements.txt
```

Dependencies: pandas, numpy, scikit-learn, PyYAML, reportlab (core); matplotlib, seaborn, pyod (for `ml_algorithms` model scripts). See `requirements.txt`.

2. **Input data:** Place `master_features_with_clustering.csv` in

`aml_detection_pipeline/data/input/`

To produce it from raw data: run the `clean_data` pipeline (see `clean_data/README.md` — run scripts 01–11 from `clean_data/scripts/` in order); then copy

`clean_data/features/final/master_features_with_clustering.csv` into `aml_detection_pipeline/data/input/`. One row per customer.

3. **Run the pipeline** using the main entry point:

```
python run_pipeline.py
```

(See [Main entry points](#) for what this does.) Outputs go to `aml_detection_pipeline/data/output/` and are copied to `task_2_model_outputs/` and `task_3_model_outputs_explanations/`.

4. **Optional:** Use the other two run scripts when needed:

- `python run_viewer.py` — explanation viewer GUI (run the pipeline at least once first).
- `python run_aml_library.py` — open the AML Knowledge Library in your browser.

How the pipeline works

The detection pipeline runs from a single input file, computes a rule-based risk score, fuses it with one or more ML anomaly scores, flags the top percentile of customers, and writes Task 2/3 outputs plus optional reports. All paths and weights are set in `aml_detection_pipeline/config/config.yaml`. The following describes each stage in order; full I/O and column contracts are in `aml_detection_pipeline/docs/CLIENT_CONTRACT.md`.

1. Input

- **File:** `aml_detection_pipeline/data/input/master_features_with_clustering.csv` (config: `paths.input_file`).
 - **Produced by:** The `clean_data/` pipeline (scripts 01–11); final artefact is `clean_data/features/final/master_features_with_clustering.csv`. Copy it into the pipeline `data/input/` directory.
 - **Requirements:** One row per customer; must include `customer_id`, a `cluster` column (string labels, e.g. `individual_0`, `business_2`), and all feature columns expected by the rule-based scorer (structuring, channel, geographic, behavioral, profile, and any optional flags). The pipeline does no merging or preprocessing; it starts from this single file.
-

2. Rule-based scoring

- **Script:** `aml_detection_pipeline/scripts/rule_based_scorer.py` (invoked by `run_pipeline.py`).
- **Input:** The same CSV as above (master features + cluster).
- **Output:** `aml_detection_pipeline/data/intermediate/rule_based_scores.csv` (config: `paths.rule_scores`).

Risk categories (aligned with the AML Knowledge Library):

Category	Config key	Default weight	Description (summary)
Structuring	<code>rule_based.category_weights.structuring_amount</code>	0.25	Round amounts, just-below-threshold patterns.
Channel	<code>rule_based.category_weights.channel</code>	0.25	High-risk channels (wire, cash-intensive, multi-channel).
Geographic	<code>rule_based.category_weights.geographic</code>	0.20	Cross-border, high-risk jurisdictions, dispersion.
Behavioral	<code>rule_based.category_weights.behavioral</code>	0.20	Unusual turnover, flow-through, lifestyle mismatch.
Profile	<code>rule_based.category_weights.profile</code>	0.10	MSB, cash-intensive business, new account, etc.
Cluster	<code>rule_based.cluster_risk_boost / cluster_secondary</code>	0.15 / 0.05	Extra weight for high-risk or secondary clusters (e.g. <code>individual_3</code> , <code>business_2</code>).

The scorer computes a 0–1 score per category from binary/flag features and weights, then sums them (including cluster boosts) into a single **rule_based_score** per customer. Output columns include `customer_id`, `structuring_risk`, `channel_risk`, `geographic_risk`, `behavioral_risk`, `profile_risk`, `cluster_risk`, and `rule_based_score`.

3. ML score discovery

- **Directory:** `aml_detection_pipeline/ml_algorithms/outputs/` (config: `paths.ml_algorithms_outputs`).
- **Convention:** The pipeline discovers every file matching `scores_*.csv` (e.g. `scores_CLEAN_LOF.csv`, `scores_ABOD_k60_pct.csv`). The suffix after `scores_` is the **model label** used in output filenames (e.g. `model_output_CLEAN_LOF.csv`).
- **Required columns:** Each CSV must have `customer_id` and exactly one numeric score column among: `score`, `scores`, `risk_score`, or `outlier_score_01`. One row per customer; customer set must match the pipeline input. No config change is needed when adding or removing score files—drop a new `scores_<model>.csv` and re-run.

4. Fusion and predictions

- **Config:** `fusion.rule_weight` (default 0.70), `fusion.partner_weight` (default 0.30), `fusion.normalize_partner_score` (default true), `predictions.top_percentile` (default 5).
 - **Process:** For each discovered `scores_<model>.csv`, the pipeline (a) min-max normalizes the ML score to [0, 1] when `normalize_partner_score` is true, (b) computes **fused_score** = `rule_weight` × `rule_based_score` + `partner_weight` × `normalized_ML_score`, (c) assigns **predicted_label** = 1 to customers in the **top 5%** by fused score, and **predicted_label** = 0 to the rest. The fused score is written as **risk_score** in the Task 2 output.
-

5. Output files (Task 2 and Task 3)

- **Per model:** For each `scores_<model>.csv`, the pipeline writes:
 - **Task 2:** `aml_detection_pipeline/data/output/model_output_<model>.csv` — columns: `customer_id`, `predicted_label` (0 or 1), `risk_score` (fused score). One row per customer.
 - **Task 3:** `aml_detection_pipeline/data/output/model_output_<model>_explanations.csv` — columns: `customer_id`, `explanation` (plain-English text, see below).
- **Copy to submission dirs:** If config sets `paths.task_2_model_outputs` and `paths.task_3_model_outputs_explanations` (e.g. `../task_2_model_outputs` and `../task_3_model_outputs_explanations`), the same files are copied there after each model so graders find them at the package root.
- **Clean run:** Each pipeline run **clears** `data/output/` (deletes all existing CSVs and PDFs) before writing, so outputs always reflect the current run.

Optional outputs (when enabled and applicable): `model_report_<model>.pdf`, `model_comparison_report.pdf`, `consistently_flagged_customers.csv`, `consistently_flagged_explanations.csv` — see `aml_detection_pipeline/docs/CLIENT_CONTRACT.md`.

6. Explanations

- **Script:** `aml_detection_pipeline/scripts/explanation_generator.py`. Uses the rule-based category breakdown (structuring, channel, geographic, behavioral, profile, cluster) and the anomaly (ML) contribution to build a short, non-technical explanation per customer.
 - **Content:** Explanations reference the same red-flag categories as the AML Knowledge Library (Task 1) and state why the customer was flagged or not; borderline cases can be noted. No jargon; suitable for investigators.
 - **Limit:** **2000 characters** per explanation (config: `explanations.max_length_chars`). Explanations are generated for **all** customers, not only those with `predicted_label=1`.
-

7. Entry point and config summary

- **Run from submission root:** `python run_pipeline.py` (which invokes `aml_detection_pipeline/scripts/run_pipeline.py` from the pipeline root).
- **Config file:** `aml_detection_pipeline/config/config.yaml` — paths, rule weights, cluster lists, fusion weights, top percentile, explanation length, PDF and viewer options. No hardcoded paths in code; all behaviour is reproducible from this file plus the input CSV and any `scores_*.csv` files present in `ml_algorithms/outputs/`.

Submission checklist (for graders)

Required deliverable	Where to find it
AML Knowledge Library and documentation	task_1_aml_library/ — README, methodology, data sources (SOURCES.md), feature-to-AML-indicator mapping (Red_Flag_to_Feature_Mapping.md). Optional web app: task_1_aml_library/AML_Library_Web/ .
Bad actor detection model output	task_2_model_outputs/ — populated when the pipeline runs. Contains model_output_<model>.csv per model (e.g. model_output_CLEAN_LOF.csv). Columns: customer_id , predicted_label , risk_score . One row per customer; top 5% by risk score are flagged (predicted_label=1). Primary model: CLEAN_LOF (fusion of rule-based + anomaly score).
Model output explanations	task_3_model_outputs_explanations/ — populated when the pipeline runs. Contains model_output_<model>_explanations.csv per model. Columns: customer_id , explanation . Plain-English, non-technical; reference AML red-flag categories (structuring, channel, geographic, behavioral, profile, cluster) and explain why each customer was flagged or not; borderline cases noted. Max 2000 characters per explanation.
Source code and trained models	aml_detection_pipeline/ , clean_data/ , task_1_aml_library/ . Model artifacts: aml_detection_pipeline/ml_algorithms/model_*/artifacts/ .
Presentation / slides / recording	To be added (e.g. in a presentation/ folder or linked in this README).

The primary model for grading is **CLEAN_LOF** (fusion of rule-based + anomaly score); its outputs are [model_output_CLEAN_LOF.csv](#) and [model_output_CLEAN_LOF_explanations.csv](#) in [task_2_model_outputs/](#) and [task_3_model_outputs_explanations/](#). Additional model outputs and comparison files are in [aml_detection_pipeline/data/output/](#).

Design and methodology (summary)

- **Task 1:** The Knowledge Library is built from authoritative AML/ATF sources (FINTRAC, FinCEN); every red flag is cited in [SOURCES.md](#). The library is structured for modelling experts (feature/pattern support) and AML investigators (reference). [Red_Flag_to_Feature_Mapping.md](#) maps red flags to data signals and engineered features used in the pipeline.
- **Tasks 2 & 3:** The detection pipeline uses a **rule-based scorer** (five risk categories aligned with the library: structuring, channel, geographic, behavioral, profile, plus cluster risk) and **unsupervised ML models** (e.g. Isolation Forest, LOF). Rule and ML scores are **fused** (e.g. 70% rule, 30% anomaly); the **top 5%** by fused score are flagged. This design addresses label scarcity (unsupervised component) and ties outputs to Task 1 (rule categories). Explanations are generated from the rule breakdown and anomaly contribution, using non-technical language and the same red-flag categories. Pipeline rule features map

to the Knowledge Library via [task_1_aml_library/Red_Flag_to_Feature_Mapping.md](#). Full input/output and score contract: [aml_detection_pipeline/docs/CLIENT_CONTRACT.md](#).

Assumptions: One row per customer in the input file; all customers receive a risk score and an explanation; paths and options are set in [aml_detection_pipeline/config/config.yaml](#) (no hardcoded paths).

Reproducibility: Same input + config + code produces the same outputs.

Extensions and traceability

- **Explainability:** Rule-based component is fully interpretable (thresholds, weights, categories). Explanations are plain language, tied to rule categories and anomaly; the viewer ([run_viewer.py](#)) lets investigators browse and search.
- **Task 1:** 147+ red flags, multiple typologies, full source traceability ([SOURCES.md](#)), feature-to-indicator mapping; optional web app for interactive use.
- **Task 2:** Multiple model outputs (rule-based + e.g. Isolation Forest, LOF variants), fusion, clustering-informed rules; primary submission pair is from one chosen model (default CLEAN_LOF); full set in [aml_detection_pipeline/data/output/](#) and in [task_2_model_outputs/](#) / [task_3_model_outputs_explanations/](#). Optional PDF reports (configurable in config).
- **Task 3:** Explanations for all customers (not only flagged); ≤2000 characters; reference red-flag categories; optional [consistently_flagged_explanations.csv](#) for customers flagged by all models.
- **Traceability:** Rule features map to AML Library categories; every output is reproducible from input, config, and code. External data or models (if any) should be listed in an [EXTERNAL_RESOURCES.md](#) in this directory.